CS 550 Operating Systems
Spring 2026

Virtualizing CPU – the mechanism

Overview

- The process abstraction.
- Three key aspects of limited direct execution
  1. Restricted ops
     - System call in OSes
  2. Switching from one process to another
     - Multiprogramming
  3. Dealing with slow ops (e.g., I/Os) in process
     - Process state transitions

Virtualizing CPU

- The OS can provide an illusion that many virtual CPUs exist (or an illusion that each instance of a running program, i.e., a process, has its own CPU).
- **Time sharing**: Running one process, then stopping it and running another.

The abstraction: process

- A *process* is an instance of a running program.
  - The process is the OS's abstraction for execution
- A process is:
  - The unit of execution
  - The unit of scheduling
  - The dynamic (active) execution context
    - vs. the program – static, just a bunch of bytes

The abstraction: process

- Process provides each program with two key abstractions:
  - Logical control flow
    - Each program seems to have exclusive use of the CPU
    - Recall the “cpu” program discussed previously.
  - Private virtual address space
    - Each program seems to have exclusive use of main memory
    - Recall the “mem” program shown in the “overview” lecture

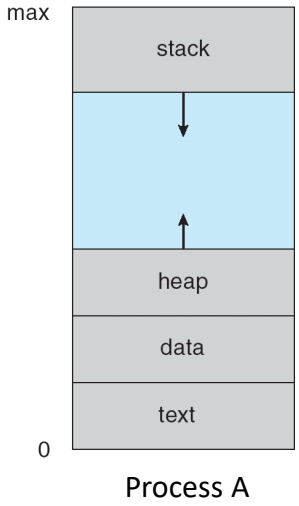
- How are these Illusions maintained?
  - Process executions interleaved (*multiprogramming*)
  - *Address spaces* managed by virtual memory system

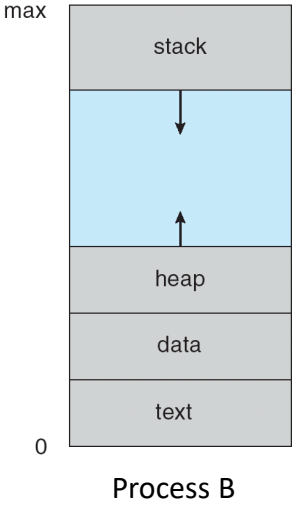
What's "in" a process?

- Logically, a process consists of (at least):
  - An address space, containing
    - the code (instructions) for the running program
    - the data for the running program (static data, heap data, stack)
  - CPU state, consisting of
    - The program counter (PC), indicating the next instruction
    - The stack pointer
    - Other general purpose register values
  - A set of OS resources
    - open files, network connections, sound channels, …
- In other words, it's all the stuff you need to run the program (or to re-start it, if it's interrupted at some point).

Address space

- (Virtual) address space:
  - Really large memory to use
  - Linear array of bytes: [0, N), N roughly 2^32, 2^64
- *Each process has it's own address space*
- Address space *enables every process to have the same view about the memory* (i.e., memory resource virtualization).
- An address space is also *a protection domain*
  - One process can't address another process's address space (without permission)
    - E.g., 0x800abcd in process p1 points to different memory than 0x800abcd in process p2
- More details when discussing memory management





Virtualizing CPU

- Goal: give each process impression it alone is actively using CPU
- Multiple concurrent processes → resources can be shared in **time** and **space**
  - Assume single uniprocessor
    - Time-sharing
    - Multi-processors: advanced issue (later)
  - Memory?
    - Space-sharing (later)
  - Disk?
    - Space-sharing (later)

Processes take turns to use the CPU. What problems can you think of when doing this?

Time-share CPU among processes

- Problems?
  1. While a process is running on the CPU, it could do something restricted (e.g., read/write other process's or even the OS's data).
     - OS needs to restrict what a process can do.
  2. A process could run forever (slow, buggy, or malicious), so other processes never get their chances to use the CPU.
     - OS needs to be able to switch between processes.
  3. A process could do something slow while using the CPU (like waiting for I/O), so the CPU is underutilized.
     - OS wants to use resources efficiently and switch CPU to other process.

Time-share CPU among processes

- OS's mechanism for processes to time-share the CPU
  - Direction execution: allow user processes to run directly on the CPU → for good performance.
  - But some limitations have to be imposed to solve the three problems previously.
  - Together, the mechanism is called limited direction execution.

How to solve three problems of time-sharing CPU among processes?

Problem 1: restricted operations

- How can we ensure user process can't harm others?
- Solution: privilege levels supported by hardware (bit of status)
  - User processes run in user mode (restricted)
  - OS runs in kernel mode (non-restricted)
    - Some instructions designated as privileged (e.g., those accessed/changed system states or critical resources), only executable in kernel mode. If they are executed in user mode, exceptions will be raised.
    - Could have many privilege levels (advanced topic)

Problem 1: restricted operations

- How can user processes, which run in user mode, perform privileged operations (e.g., requesting device resources)?
  - System calls (function calls implemented by OS) which change CPU"s change privilege level on invocations.

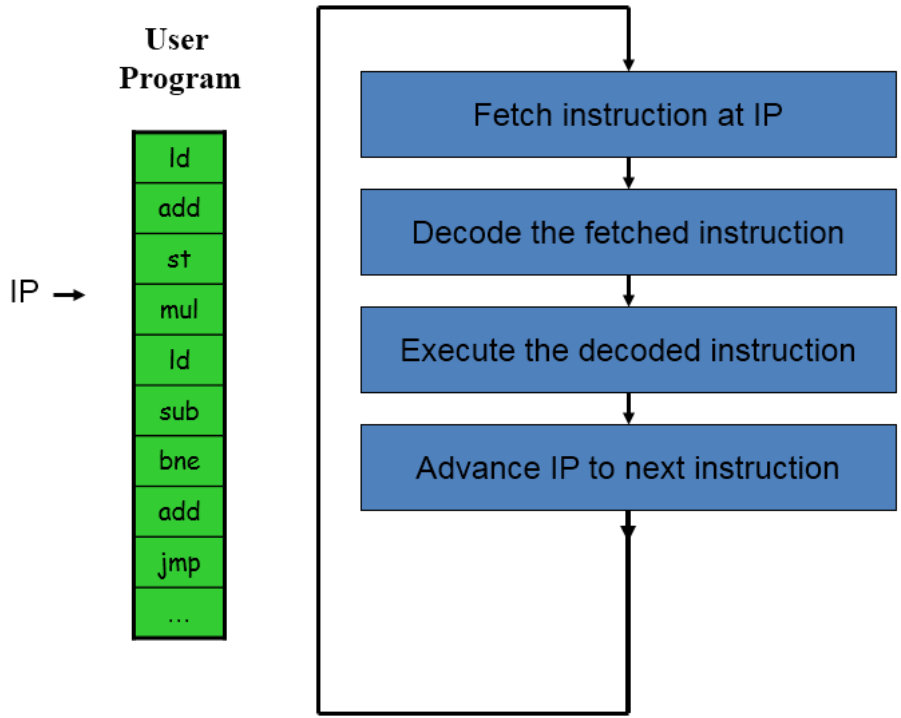
System calls

- A type of special "protected procedure calls" *allowing user-level processes to request services from the kernel*.
- System calls provide:
  - An abstraction layer between processes and hardware, allowing the kernel to provide access control, arbitration
  - A virtualization of the underlying system
  - A well-defined interface for system services

System calls vs. C library functions

- Each system call has a corresponding standard C library wrapper routines, which hide the details of system call entry/exit.
  - sys_brk() → malloc() (<stdlib.h>)
- Is the reverse true?
  - strlen() (<string.h>) ? → all in user space
  - open() (<fcntl.h)? → sys_open()
  - printf() (<stdio.h>)? → write() → sys_write()
  - sprintf() (<stdio.h>)? → all in user space

A primer on interrupt – CPU's 'fetch-execute' cycle

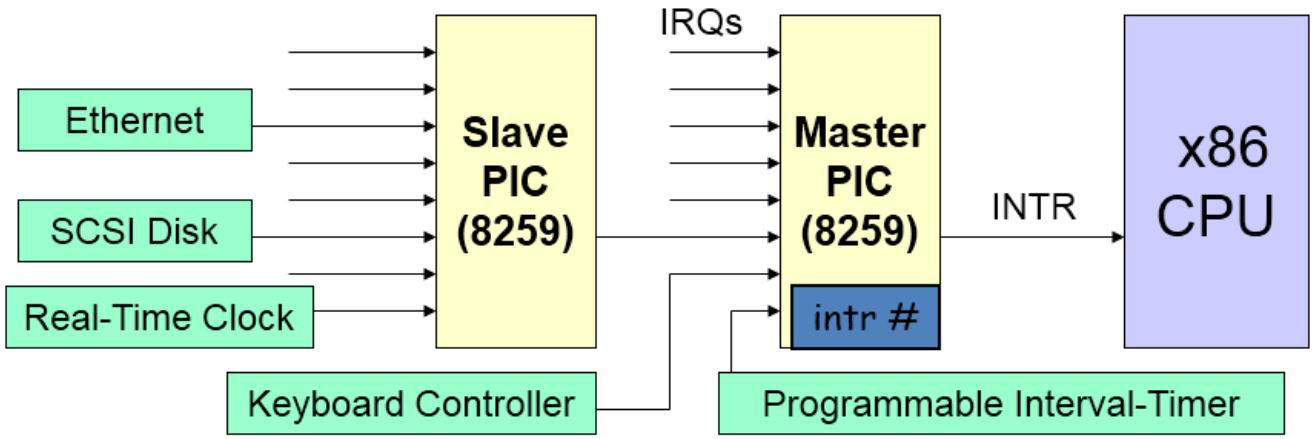


Q: How can external devices notify the CPU about certain events?

A: Through the usage of interrupts

IP: Instruction Pointer (or Program Counter, PC)

A primer on interrupt – Interrupt hardware (legacy systems)

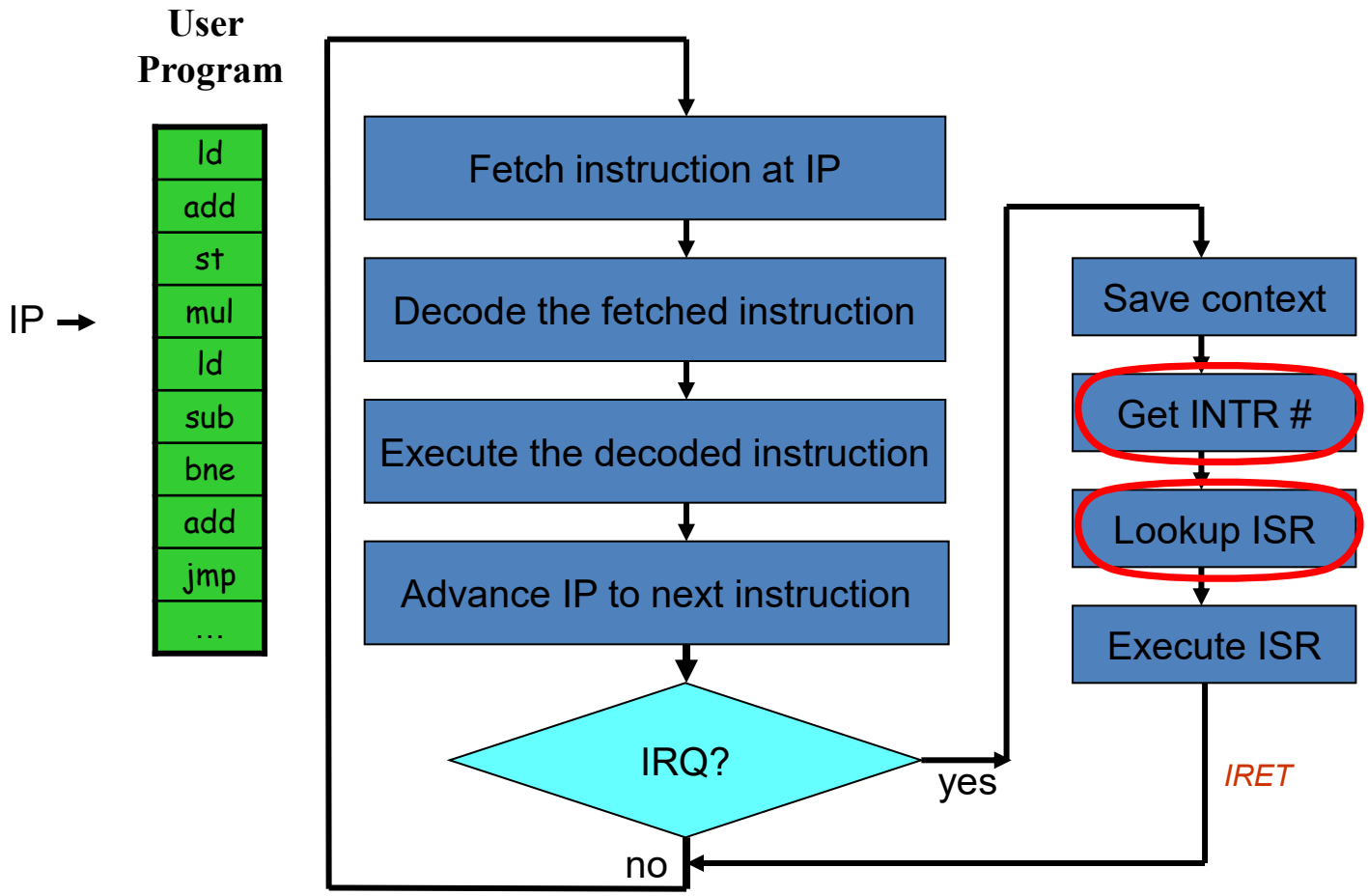


- I/O devices have (unique or shared) Interrupt Request Lines (IRQs)
- IRQs are mapped by special hardware to interrupt numbers, and passed to the CPU
  - This hardware is called a *Programmable Interrupt Controller (PIC)*

A primer on interrupt – the Programmable Interrupt Controller (PIC)

- Responsible for telling the CPU when a specific external device wishes to ‘interrupt’
  - Needs to tell the CPU which one among several devices is the one needing service
- PIC translates IRQ to interrupt number
  - Raises interrupt to CPU
  - Interrupt # available in a register

A primer on interrupt – CPU's 'fetch-execute' cycle with interrupt



IRQ: Interrupt Request

ISR: Interrupt Service Routine, a.k.a. Interrupt handler

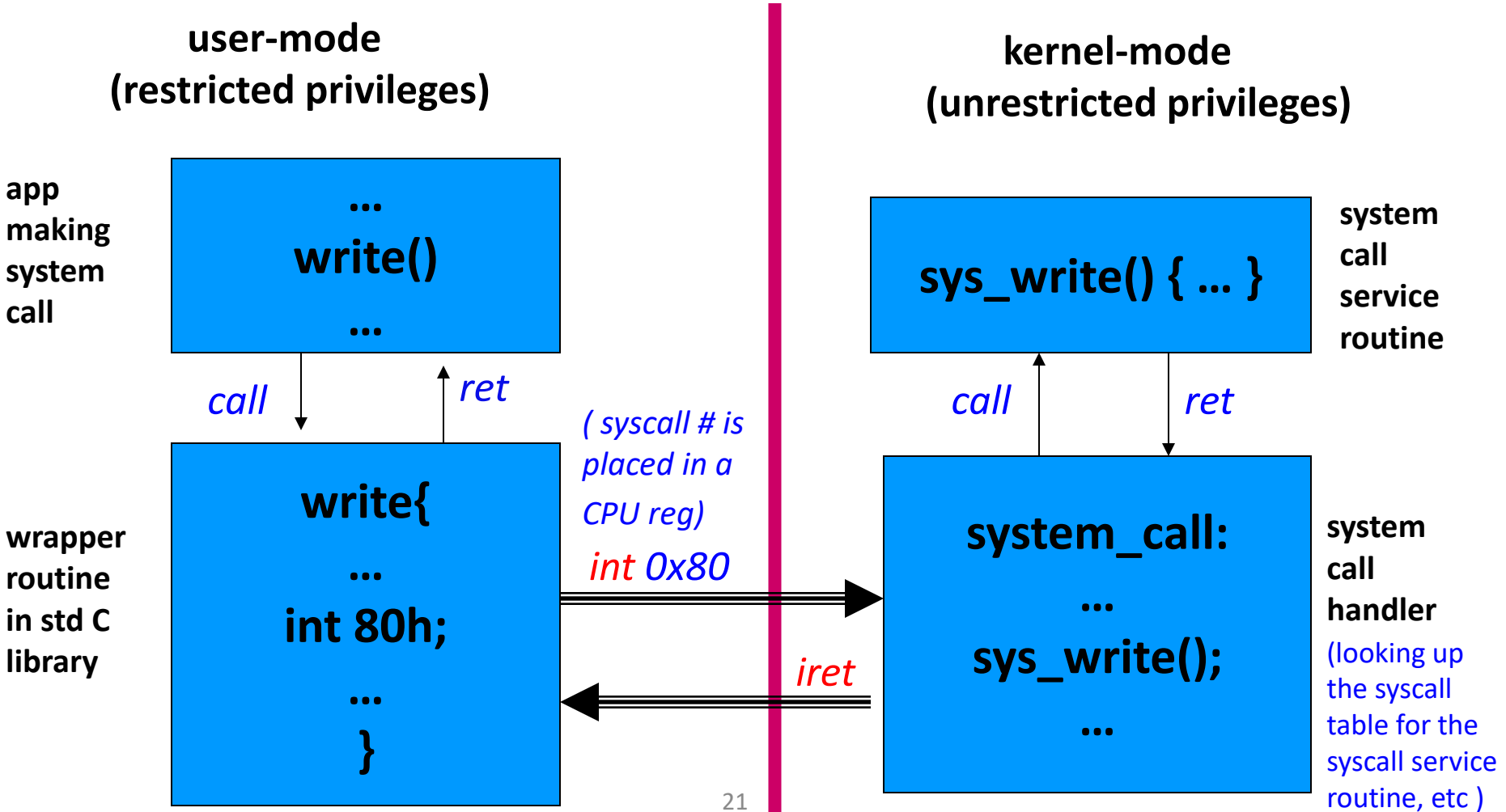
A primer on interrupt – Interrupt Descriptor Table (IDT)

- The ‘entry-point’ to the interrupt-handler is located via the Interrupt Descriptor Table (IDT)
- *Interrupt Service Routine* = IDT[Interrupt number]
  - Also called *interrupt handler*
- IDT is in memory, initialized by OS at boot
- How to locate base of IDT?
  - CPU has a register, *idtr*, pointing to IDT, initialized by OS via the LIDT (x86) instruction at boot

A primer on interrupt – same interrupt mechanism used for other control transfers

- We've seen Interrupts: raised externally by device
- Traps (or exceptions): raised internally by CPU
  - 0: divide-overflow fault
  - 3: breakpoint
  - 6: Undefined Opcode
  - 13: General Protection Exception
- System call can be implemented this way too
  - Linux system call: int 80h
    - "int" instruction generates a "software interrupt" or "trap", causing the CPU transition from user mode to kernel mode. 80h is the interrupt ID.

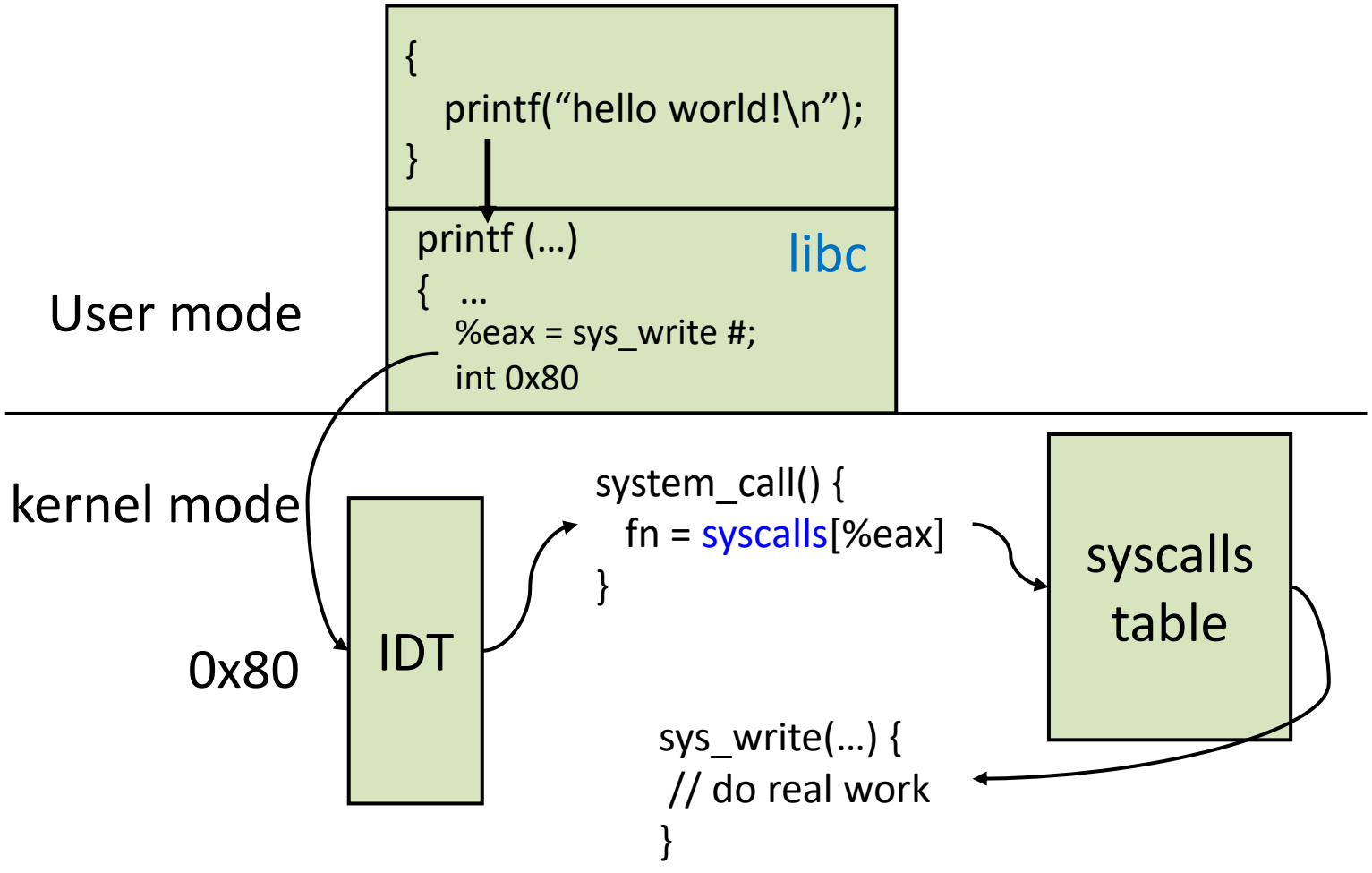
Invoking system calls



Invoking system calls: more details

- In user program
  - call the library function that uses a system call
- In library function
  - Preparation work
  - Save the syscall number in %eax (x86)
  - Call "int 80h" (Linux)
- Hardware: locate the system call trap handler using the interrupt ID 80h
- In trap handler:
  - Save user process context
  - Look up the intended system call in the "system call table"
- In system call:
  - Perform the requested service
  - Return to user mode by "iret" instruction, which restores the user process context

Invoking system calls: another view



The system-call jump-table (system call table)

- Any specific system-call is selected by its ID-number (i.e., the *system call number*, which is placed into register %eax)
- An array of function-pointers is directly accessed (using the ID-number)
- This array is named '**sys_call_table**[]' in Linux.

Syscall return values

- Recall that library calls return -1 on error, and place a specific error code in the global variable errno
- System calls return *specific negative values* to indicate an error
  - Most system calls return –errno
  - On x86, the return value is put into %eax, so that the library wrapper function can access.
- The library wrapper function is responsible for conforming the return values to the errno convention

System call argument passing

Three general methods used to pass arguments to the OS:

- Method 1: pass the arguments in registers (simplest)
  - Any drawbacks?
- Method 2: arguments are placed, or pushed, onto the stack by the program and popped off the stack by the OS kernel code (i.e., the syscall implementation)
- Method 3: arguments are stored in a block, or table, in memory, and address of block passed as a parameter in a register
  - This approach taken by Linux and Solaris
- Which method does xv6 use?

Problem 2: How to take CPU away

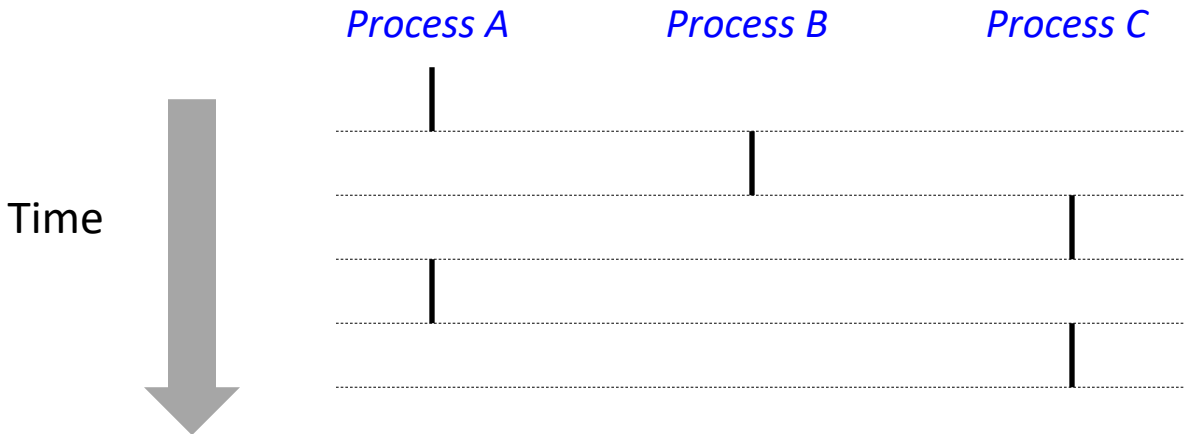
- When a process is running on the CPU, it could be running forever.
  - Slow
  - Buggy
  - Malicious
- How can OS take CPU away from one process and give it to another one?
- Or more generally, what are the requirements for OS to support multiprogramming?

Multiprogramming

- All OS support processes, but different number of processes
  - process = job
- *Uniprogramming (or unitasking)*: only one process at a time – a process runs from start to end, no other processes can start when it is running
  - Example: early systems, MSDOS
  - Good: simple
  - Bad: poor resource utilization, inconvenient for user
- *Multiprogramming (or multitasking)*: multiple processes at a time, when one process waits (e.g. I/O), switch to another → *interleaved process execution*
  - Example: Unix, Linux, Windows NT, solaris, all modern OS
  - Good: increase utilization, user convenience
  - Bad: OS complex
  - NOTE: different from multiprocessing (systems with multiple processors)

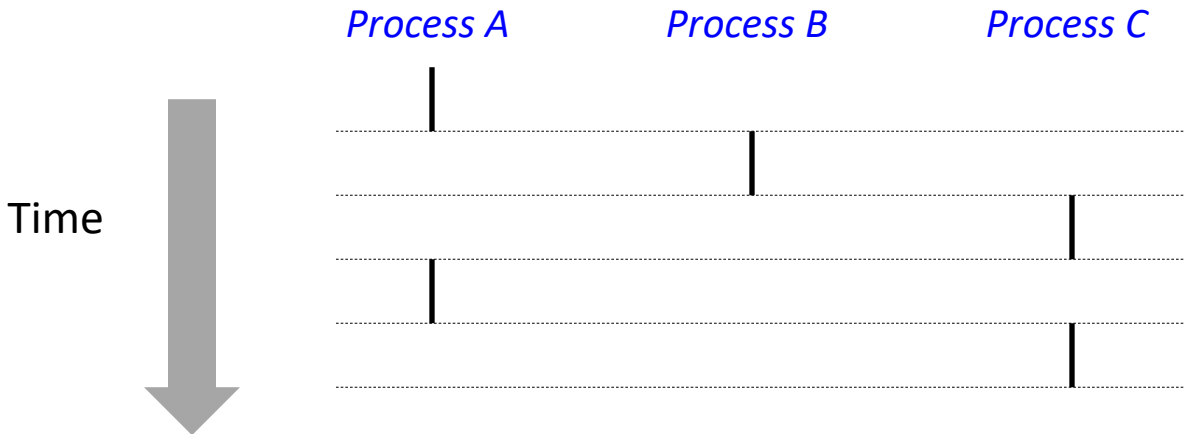
Multiprogramming

- Two processes *run concurrently* (*are concurrent)* if their flows overlap in time
- Otherwise, they are *sequential*
- Examples: which pair(s) is/are concurrent/sequential? (running on single core):



Multiprogramming

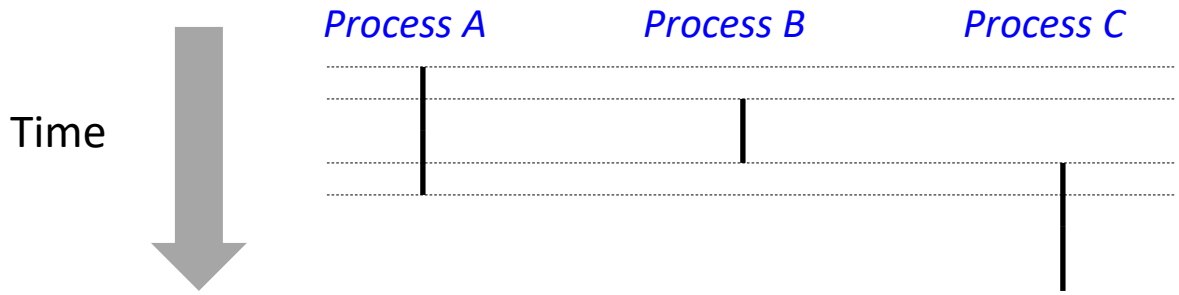
- Two processes *run concurrently* (*are concurrent)* if their flows overlap in time
- Otherwise, they are *sequential*
- Examples: which pair(s) is/are concurrent/sequential? (running on single core):
  - Concurrent: A & B, A & C
  - Sequential: B & C



Multiprogramming

User view of concurrent processes:

- Control flows for concurrent processes are physically disjoint in time
- However, we can think of concurrent processes are running in parallel with each other



Multiprogramming

- OS requirements for multiprogramming
  - *Policy*: what process to run? (later)
  - *Mechanism*: how to switch process?
  - Methods to protect process from one another (memory management, later)

- Separation of *policy* and mechanism
  - Recurring theme in OS
  - *Policy*: decision making with some performance metric and workload
    - Scheduling (later)
  - *Mechanism*: low-level code to implement decisions
    - Dispatching (today)

Process dispatching loop

Q1: how does OS gain control (i.e., how to initiate the transfer)?

```
while(1) {
    cur_proc runs for a while; // in user space
    control transferred to kernel space;
    save cur_proc's state (i.e., execution context); // in kernel space;
    new_proc = schedule(ready processes); // in kernel space
    load new_proc's state; // in kernel space
    cur_proc = new_proc; // in kernel space
    control transferred to user space (initiated by kernel);
}
```

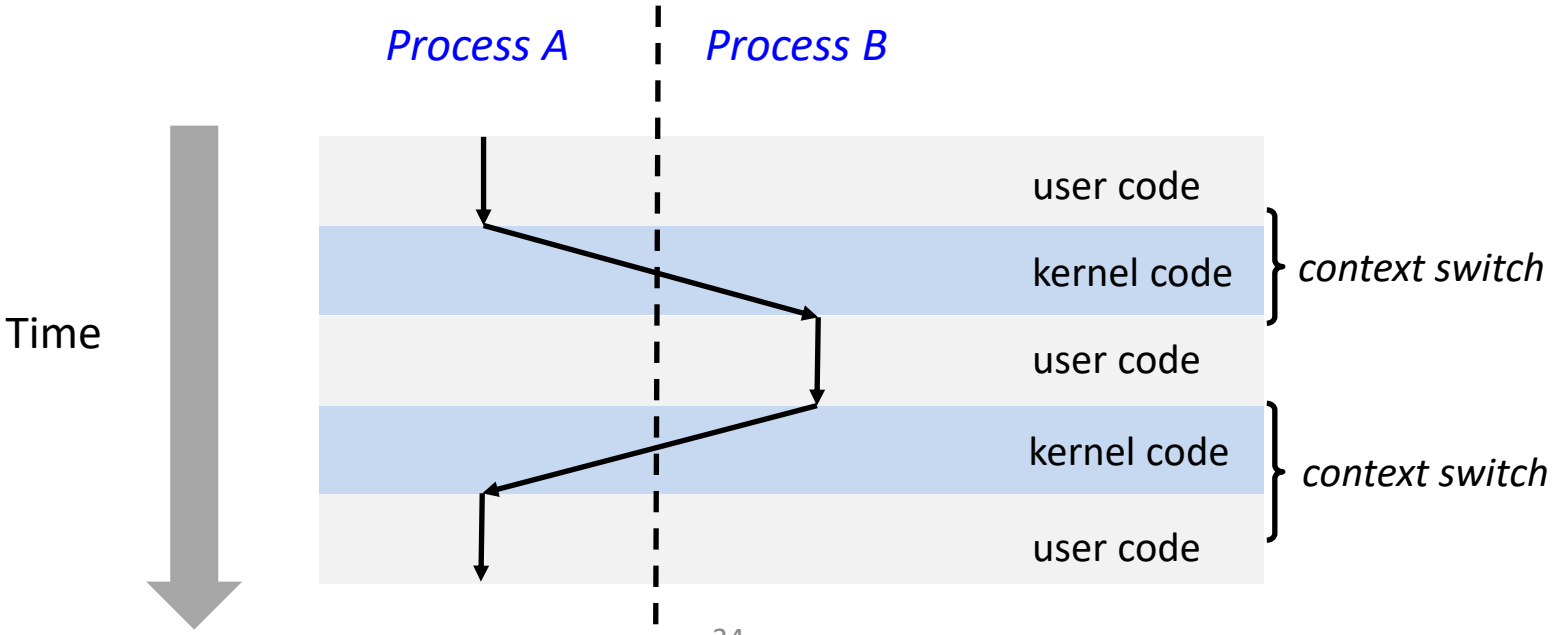
Q2: what state should be saved?

Q3: Where to find these processes?

context switch

Context switch

- Processes are managed by OS kernel
- Control flow passes from one process to another via a *context switch*



Q1: How does OS gain control?

- User process is in the “fetch-execute” cycle
- Two ways OS gains control
  - Hardware interrupts: external events
    - E.g. network card, disk controller, timer
    - OS gains control via Interrupt Service Routine (ISR)
  - Traps or exceptions: generated by instructions running inside CPU
    - E.g. system calls, errors, page faults
    - OS gains control via trap/exception handlers

Q1: How does kernel gain control?

We now know external events or system calls can trigger control transfer from user space to kernel space. But what if there is no external events or system calls, will the running process run forever?

Q1: How does kernel gain control?

Two approaches to gain control in this case

- *Cooperative approach*: OS trusts processes to voluntarily yield control back (e.g., when finish, when calling system calls).
  - Is it good?
- Non-cooperative approach: OS preempts processes by periodic alarm clock (external events!)
  - Dispatcher gains control on every hardware timer interrupt (CPU or other chip)
    - E.g., By default programmed to 1 ms in Linux on x86 CPU
  - Processes are assigned time slices
  - Dispatcher counts timer interrupts to decide when to schedule another process.
  - Dispatcher also gets control upon every system calls or other interrupts. In other words, as long as control is transferred to OS, the dispatcher (i.e., the scheduler) gets to run.
  - Why good?

Q2: What state must be saved?

- In a context switch, the execution context of a process *P* must be saved, so that when the scheduler gets back to the process *P*, it can restore this state and continue.
- *What to save*? (i.e., what does the process execution context consist of?)
  - Any the CPU registers that *P* may be using. For example
    - Pointer register (EIP, ESP, EBP)
    - General register (EAX, EBX, …)
    - Control register (CR3, …)
      - The CR3 register records the base address of the page table of the process
    - …
- *Where to save*?
  - On every interrupt or trap, save state in **Process Control Block (PCB)**

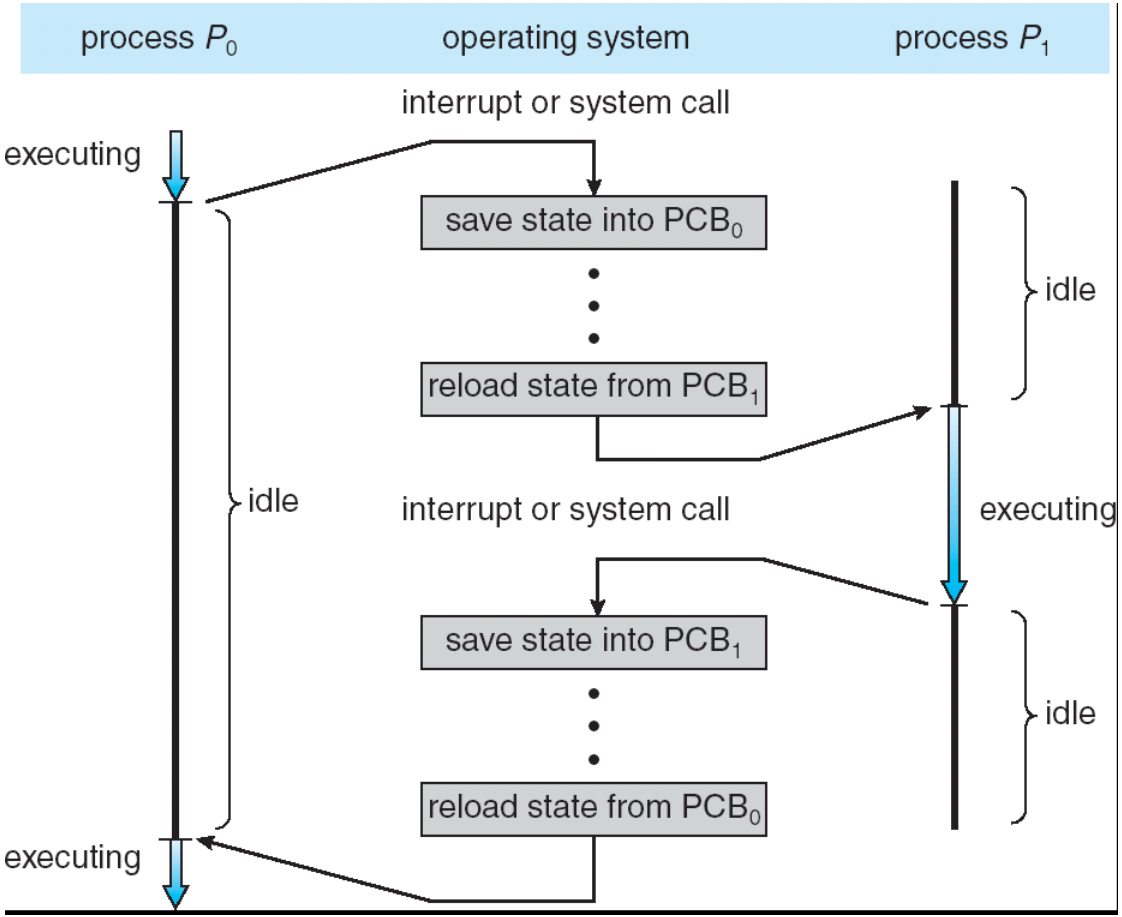
From the loader to the kernel

- Once a program is loaded, the kernel must manage this new process
- The OS maintains a data structure to keep track of a process's state
  - Called the process control block (PCB) or process descriptor
  - Identified by the PID
- The OS kernel has a data structure, called a process table, to manage all the PCBs (each entry of the table is a PCB).

Process control block (PCB)

- In modern OS, the PCB is a data structure with many fields:
  - process ID (PID)
  - parent process ID
  - process state
  - physical address of the process's page directory
  - UNIX user id, group id
  - scheduling priority
  - accounting info
  - program counter, stack pointer, registers
  - … …
- In Linux:
  - defined in **`struct task_struct`** **(include/linux/sched.h)**
  - over 95 fields!!!
- In xv6
  - defined in **`struct proc`** **(proc.h)**
  - only 13 fields!!!

Context switch – all together



Q3: Where to find processes?

- Data structure: process scheduling queues
  - Job queue – set of all processes in the system
  - Ready queue – set of all processes residing in main memory, ready and waiting to execute
  - Device queues – set of processes waiting for an I/O device
- On xv6, only the ready queue is supported.

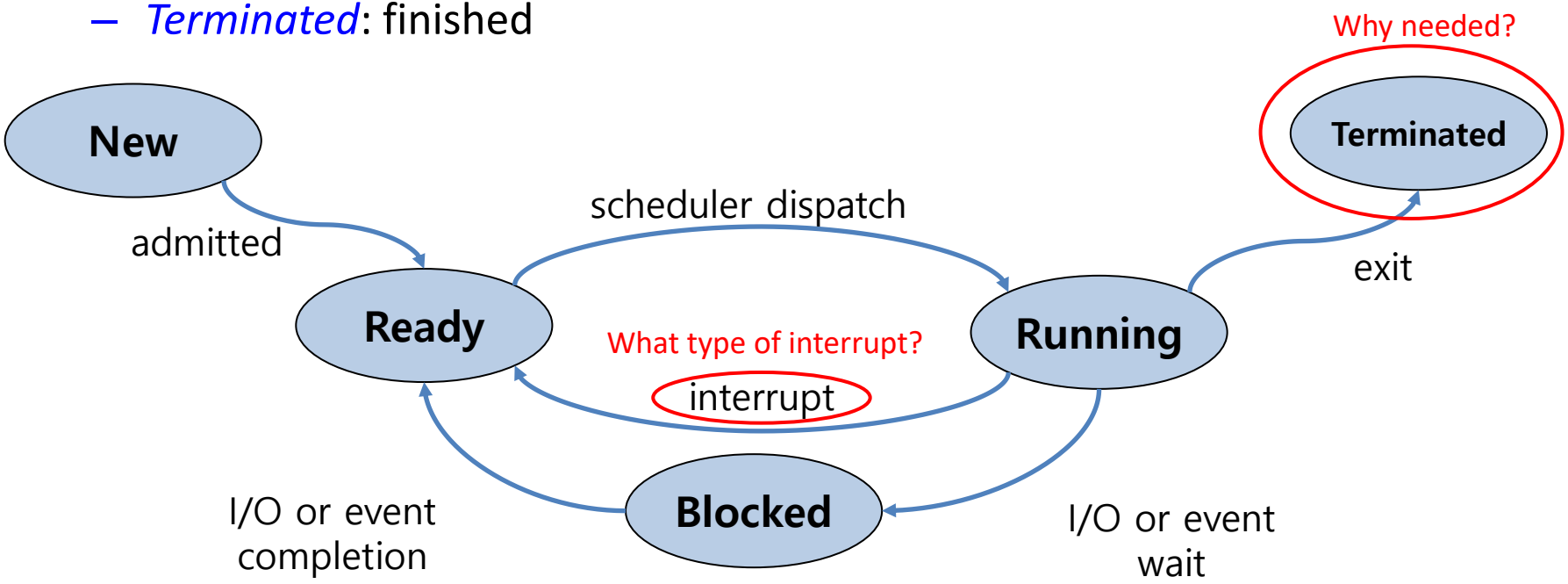
- Processes migrate among the various queues when their states change

Problem 3: How to deal with slow ops, such as I/Os, in processes

- Solution: when running process performs op that does not use CPU, OS switches to process that needs CPU.
- Process state transitions.

Process state diagram

- Process states
  - *New*: being created
  - *Ready*: waiting to be assigned a CPU
  - *Running*: instructions are running on CPU
  - *Blocked*: waiting for some event (e.g. IO)
  - *Terminated*: finished



Time-share CPU among processes: summary

- The three problems:
  1. While a process is running on the CPU, it could do something restricted (e.g., read/write other process's or even the OS's data).
     - OS needs to restrict what a process can do.
       - » Different privilege modes in CPU
       - » System calls
  2. A process could run forever (slow, buggy, or malicious), so other processes never get their chances to use the CPU.
     - OS needs to be able to switch between processes.
       - » Multiprogramming, context switches
  3. A process could do something slow while using the CPU (like waiting for I/O), so the CPU is underutilized.
     - OS wants to use resources efficiently and switch CPU to other process.
       - » Process states and state transition

Summary

- CPU virtualization: context switching gives each process impression it has its own CPU
- Direct execution makes processes fast
- Limited execution at key points to ensure OS retains control
- Hardware provides a lot of OS support
  - user vs kernel mode
  - timer interrupts
  - automatic register saving